

SIMTiX: A Compact Open-Source SIMT Accelerator for RISC-V with a Pipelined FP32/FP16 Unit and an Interconnect-Bound Timing-Closure Study

Krishna Gorai

Indian Institute of Technology Mandi

Email: kg05112001@gmail.com

Abstract—We present SIMTiX, a compact, fully-open Single-Instruction Multiple-Thread (SIMT) accelerator for RISC-V whose entire path—RTL, simulation, synthesis, place-and-route, and post-route gate-level sign-off—is reproducible on one workstation. SIMTiX attaches eight lanes grouped into warps to a five-stage RISC-V host through a memory-mapped offload interface, and implements the full SIMT toolbox: warp scheduling with a background memory engine for latency hiding, address coalescing, a per-warp reconvergence stack, and an on-chip scratchpad. Placing the vector register file and scratchpad in distributed RAM (LUTRAM) shrinks the design $5.7\times$ and closes 100 MHz timing. Every lane gains a pipelined floating-point unit in single and half precision (FP32/FP16); half precision reuses the single-precision datapath through NaN-boxed widen-compute-narrow, shown to be correctly single-rounded with no extra cycles. The unit covers add, multiply, the fused multiply-add, and iterative divide and square root, verified bit-exact against an IEEE-754 reference over 116,089 vectors. Our central result is a *placed* timing-closure study of this FP datapath on a Zynq UltraScale+ FPGA. The FP-enlarged fabric ($2.5\times$ the integer LUTs) is interconnect-bound: a two-stage FMA closes at only 78 MHz, routing-dominated. De-congestion confirms the diagnosis—sharing one serial divide/square-root unit lifts the placed maximum to 84 MHz with the logic unchanged—and a lane-count design-space exploration shows the limit is a single lane’s FMA cone, not the fabric size. A three-stage FMA *meets* 100 MHz and moves the critical path off the floating-point datapath; a final fetch/execute split of the warp scheduler carries the closure to the complete SoC, signed off at 103.7 MHz as a deployable, pin-constrained bitstream with a warning-free DRC. The complete RTL, benchmark harness, and FPGA flow are public.

Index Terms—SIMT, GPGPU, RISC-V, hardware accelerator, floating-point, half precision, fused multiply-add, timing closure, FPGA, open-source hardware.

I. INTRODUCTION

Graphics processing units execute thousands of scalar threads in lockstep groups called *warps* under a Single-Instruction Multiple-Thread (SIMT) model. The mechanisms that make this efficient—warp scheduling for latency hiding, memory coalescing, control-divergence handling, and on-chip scratchpads—are well documented for commercial GPUs [7],

[8] but hard to *study in RTL*. Production GPUs are proprietary; open GPGPU cores such as Vortex [1] are powerful but large and complex to bring up; and small academic cores are often simulator-only or stop short of a real implementation flow.

This paper presents **SIMTiX**, a deliberately compact open-source SIMT accelerator for RISC-V whose entire path—RTL through bitstream and post-route gate-level sign-off—is reproducible on a single workstation with free or widely available tools. SIMTiX is small enough to read end-to-end yet implements the complete SIMT mechanism set, and we extend each lane with a pipelined floating-point unit. Its headline result is a *placed* timing-closure study showing that, once the lanes carry floating point, the design becomes *interconnect-bound* rather than logic-bound, and we close 100 MHz by attacking the right lever—the per-lane fused multiply-add microarchitecture—identified through a measured diagnose-confirm-localize sequence.

Contributions.

- A compact, fully-open SIMT accelerator (eight lanes, four resident warps) integrated with a five-stage RISC-V host as a self-contained system-on-chip, exercising the entire RTL-to-bitstream flow (Sections II–III).
- An implementation result showing that moving the vector register file and scratchpad into distributed RAM reduces LUT usage by $5.7\times$ and closes 100 MHz timing, with a clear analysis of why the register-file read multiplexer, not the ALUs, dominates a naive SIMT datapath (Section III).
- A per-lane floating-point extension covering both FP32 and FP16, in which half precision is obtained almost for free from the single-precision path via NaN-boxed widen-compute-narrow (shown to be correctly single-rounded), verified bit-exact over 116,089 vectors (Section V).
- A placed FPGA timing-closure study that diagnoses the FP-enabled design as interconnect-bound, confirms it by de-congestion, localizes the limit to a single lane’s FMA cone via a lane-count design-space exploration, closes 100 MHz with a three-stage FMA that moves the critical

path off the FP datapath, and carries the closure to the complete SoC—signed off at 103.7 MHz as a deployable bitstream with full pin constraints and a warning-free DRC (Section VI).

II. SIMTiX ARCHITECTURE AND PLATFORM

A SIMT machine groups scalar *threads* into *warps* that share one instruction stream and execute on a vector of *lanes*. SIMTiX uses a warp size equal to its lane count ($\text{WARP_SIZE} = \text{NUM_LANES} = 8$), with $\text{NUM_WARPS} = 4$ warps resident and time-sharing a single fetch/decode/execute datapath; when one warp stalls on memory, the scheduler issues another, hiding latency. Unlike a packed-SIMD vector unit such as the RISC-V “V” extension [13], where one instruction operates on a register-resident vector, SIMTiX keeps a scalar thread per lane with an independent program counter abstraction managed by the reconvergence stack, so data-dependent control flow is handled in hardware rather than by masking in software.

SIMTiX is an offload accelerator. The host programs a command block over a memory-mapped (MMIO) interface—kernel PC, base pointers, thread count N —writes a GO bit and polls DONE; on GO the accelerator launches $\lceil N/8 \rceil$ warps, seeding each thread’s id and arguments through registers at spawn. Four mechanisms make up the platform. *Warp pool and scheduler*: four resident slots, each with its own PC, active mask, register-file partition, and reconvergence stack, share one datapath and one data port; a round-robin scheduler issues one runnable warp per cycle and a *background memory engine* services a memory access while other warps’ compute instructions keep the datapath busy. *Coalescing*: the data port is one cache line wide ($\text{LINE_WORDS} = 8$, 256 bits); the memory engine groups active lanes’ addresses by line and spends one transaction per distinct line, so a contiguous warp access collapses to a single transaction and a scattered one costs up to eight. *Reconvergence stack*: each warp owns a SIMT stack whose top-of-stack frame defines the live PC and active mask; a branch on which active lanes disagree pushes a frame so the two lane groups run in turn and reconverge at a join PC, supporting compiler-free single-sided *if* and divergent loops. *Scratchpad*: a per-warp 64-word aperture serviced from on-chip RAM lets data reused across a warp’s lanes be staged once, cutting global traffic. The active mask additionally drives a per-lane clock-enable, so masked-off lanes are not clocked; the unused-lane fraction is dynamic energy a divergence-aware design saves (up to 22.5% on a divergence-heavy kernel) [10]. Every mechanism is instrumented with hardware counters so it can be measured directly in RTL, rather than estimated in a software model [9].

III. IMPLEMENTATION

SIMTiX is written in synthesizable SystemVerilog and integrated with a reused five-stage RV32I host into a self-contained chip (*chip_top*, Fig. 1): host core, driver ROM, accelerator, and an eight-bank on-chip shared memory. In the integrated chip the host loads operands into shared memory,

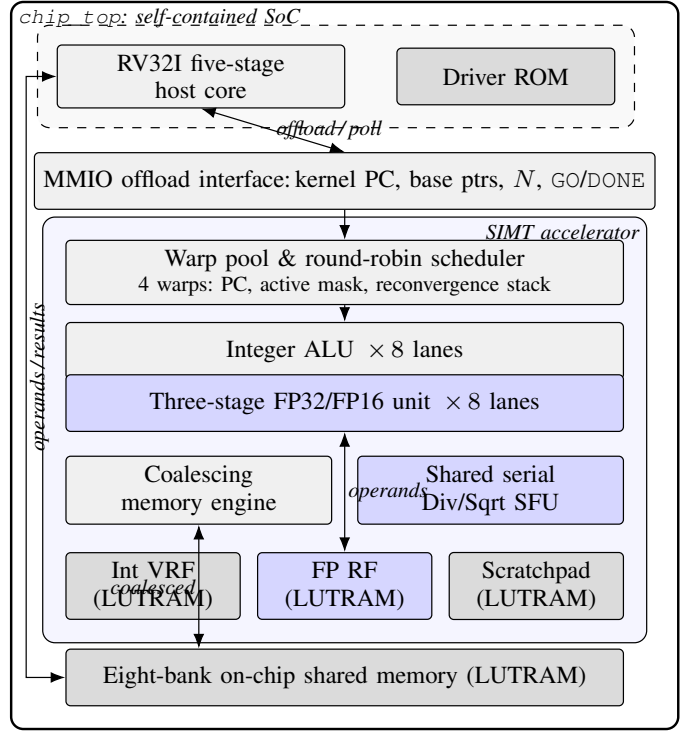


Fig. 1. SIMTiX system architecture. The five-stage RISC-V host drives the SIMT accelerator over a memory-mapped offload interface and exchanges operands and results through the eight-bank on-chip shared memory; the whole loop is one self-contained *chip_top*. Floating-point blocks (per-lane pipelined FP32/FP16 unit, FP register file, and the shared serial divide/square-root SFU) are tinted blue. Register files and the scratchpad live in distributed RAM (LUTRAM).

launches the accelerator, and reads results back—the whole CPU→memory→accelerator→memory→CPU loop runs in hardware.

From flip-flops to LUTRAM. The first synthesis of the accelerator on a Zynq UltraScale+ (*xczu7ev*) consumed 170,868 LUTs—74% of the device—at only 10% of its flip-flops. The cause is structural: a flip-flop vector register file (VRF) of $4 \times 8 \times 32$ 32-bit words is 32,768 registers read as *vrf[warp][lane][reg]* with *both* warp and register variable, i.e. a 128-to-1 multiplexer per lane per read port. That multiplexer fabric, not the ALUs, dominates area and sets the critical path. Re-expressing the VRF as one distributed-RAM bank per lane (and likewise the scratchpad) removes essentially all those registers: LUTs fall $5.7\times$ to 30,088 (13%), flip-flops to 4,590 (1.0%), and the design closes 100 MHz timing with positive slack at 0.81 W. The complete chip, taken through *opt/place/phys-opt/route* to a bitstream, meets all timing at 100 MHz and uses 34,850 LUTs (15.1%), 6,130 flip-flops, 24 DSP, and zero BRAM at 1.09 W; the accelerator is $\sim 87\%$ of the logic.

Verification ladder. Correctness is established at four levels: cycle-accurate RTL simulation with self-checking testbenches (Verilator [18]); behavioral simulation in the vendor GUI; full place-and-route with post-route timing sign-off; and post-implementation *gate-level* functional simulation of

TABLE I
MEASURED CYCLES AND SPEEDUP OVER THE FIVE-STAGE RV32I HOST
RUNNING IDENTICAL WORK (LARGEST SIZE; `collatz` AT $N=256$).
LANE UTILIZATION FROM THE ACCELERATOR COUNTERS.

Kernel	N	Accel	CPU	Speedup	Lane-util
<code>vadd</code>	1024	1,413	13,322	$9.43\times$	100%
<code>saxpy</code>	1024	1,669	16,394	$9.82\times$	100%
<code>fir</code>	1024	2,053	14,345	$6.99\times$	100%
<code>relu</code>	1024	1,284	12,809	$9.98\times$	93.7%
<code>collatz</code>	256	9,637	26,601	$2.76\times$	68.3%
<code>reduce</code>	1024	6,023	4,363	$0.72\times$	39.8%

the routed netlist (UNISIM primitives), which reproduces the integrated chip’s golden checksum of 964 bit-for-bit. The benchmark suite below adds a fifth, breadth-oriented level.

IV. PLATFORM EVALUATION

We characterize the integer platform with six kernels chosen to stress distinct mechanisms: `vadd` ($C_i = A_i + B_i$, streaming), `saxpy` ($3A_i + B_i$, coalescing), `fir` ($A_i + 2A_{i+1} + A_{i+2}$, stencil), `relu` (light divergence), `collatz` (heavy data-dependent divergence), and `reduce` (scratchpad reduction). Each is assembled from RISC-V source with a stock toolchain, run at $N \in \{8, 64, 256, 1024\}$ threads, and checked bit-exact against a golden model. For an apples-to-apples speedup, each kernel is also written as a scalar loop and run on the *same* five-stage host in RTL.

Table I gives the headline platform result. Throughput ramps with occupancy (latency hiding) and saturates near 6.7 work-items/cycle—below the eight-lane ideal because of the single shared data port. Streaming kernels reach 7–10 $\times$ over the scalar host; `fir` is lower (7.0 $\times$) due to its stencil’s partial coalescing; `collatz` achieves 2.8 $\times$ as heavy divergence erodes—but does not eliminate—the SIMT advantage; and `reduce` falls below break-even (0.72 $\times$), reported honestly—a single-lane collapse on a single-ported scratchpad is slower than the scalar core and wants a dedicated cross-lane reduction primitive. SIMTiX is parameterized in its lane and warp counts; a sweep over $\text{NUM_LANES} \in \{4, 8, 16, 32\}$ and $\text{NUM_WARPS} \in \{2, 4, 8\}$ (all twelve elaborate and verify bit-exact) shows throughput scaling near-linearly with lanes ($\sim 1.97\times$ per doubling) while the resident-warp count makes little difference under the single-cycle behavioral memory model—occupancy beyond two warps pays off only under realistic multi-cycle memory.

V. FLOATING-POINT EXTENSION

The base engine is integer-only. We extend it with floating point in both single (FP32) and half (FP16) precision, keeping the same SIMT structure: the extension is a per-lane execute unit and a second per-lane register file, not a new pipeline. Each lane gains a compact fixed-latency FP unit beside its integer ALU, and a second per-lane register file holds the 32 floating-point registers in distributed RAM, addressed by {warp, register} exactly like the integer VRF, with an independent write port so an FP result and an integer load can retire

together. The unit implements add, subtract, multiply, the fused multiply-add (all four sign variants), sign-injection, min/max, the ordered compares, integer $\leftrightarrow$ float and FP32 $\leftrightarrow$ FP16 conversion, the bit-moves, and classify, in both precisions; rounding is round-to-nearest-even per IEEE 754 [14] and subnormals are flushed to zero, a throughput-oriented choice that keeps each lane small.

Half precision via widen–compute–narrow. A 16-bit half lives in a 32-bit register NaN-boxed (upper 16 bits all ones); a mis-boxed value reads as a canonical NaN per the RISC-V convention [12]. For an FP16 operation the unit widens the operand exactly to FP32, runs the single-precision datapath, and rounds back to FP16. This is correct, not approximate: because the FP32 intermediate carries a 24-bit significand and $24 \geq 2 \cdot 11 + 2$, rounding first to FP32 and then to FP16 equals rounding directly to FP16 [17], so FP16 add/sub/mul, the fused multiply-add, and conversions are correctly single-rounded. Half precision thus reuses the verified single-precision logic almost entirely, adding only the widen, narrow, and box steps—and, as Fig. 2 shows, no extra cycles.

Fused multiply-add. The FMA computes $\pm(a \cdot b) \pm c$ with a *single* rounding [15], so it cannot be built by chaining the multiplier and adder. The unit forms the exact 48-bit product, anchors it in a wide fixed-point accumulator, aligns the addend while capturing a sticky bit, and normalizes and rounds once; when the addend out-ranges the product beyond the rounding window the product is provably below half a ulp, giving the addend exactly and bounding the alignment shifter. It has fixed latency and commits through the ordinary writeback path; Section VI describes how this combinational chain is pipelined for timing closure.

Multi-cycle divide and square root. Division and square root are produced one quotient/root bit per cycle by a digit-recurrence [16]—the engine’s only multi-cycle operations. Because such a core is comparatively large, the engine has a *single shared* one: when a warp issues a divide or square root the engine captures all its lanes’ operands and parks the warp on a stall scoreboard, then sequences the active lanes through the shared core one at a time—the same park-and-resume mechanism the memory engine uses, so the scheduler keeps issuing other warps onto the idle datapath meanwhile. At most one warp’s divide/square root is in flight, trading latency (k active lanes take $\sim k$ times as long, hidden across warps) for the area saving quantified in Section VI.

Verification. Each unit is verified standalone against a DPI-C IEEE-754 reference: FP32 against the host’s correctly-rounded float operations (including `fmaf` and `sqrtf`), and FP16 against a double-based reference cross-checked with `_Float16`. Over random vectors spanning the product-dominant, balanced (cancellation), and addend-dominant regimes plus directed infinity/NaN/zero/flush/overflow, divide-by-zero, square-root-of-negative, mis-boxed-operand, and conversion corners, the pipelined unit passes 96,066 checks and the iterative divide/square-root unit a further 20,023, all bit-exact (Table II). End-to-end FP kernels then run the full warp pool: streaming add and multiply-add in each precision, a

TABLE II

FLOATING-POINT OPERATION COVERAGE. EVERY OPERATION IS IMPLEMENTED AND CHECKED IN BOTH FP32 AND FP16; ALL CHECKS ARE BIT-EXACT AGAINST AN IEEE-754 REFERENCE.

Category	Operations (each in FP32 and FP16)
Arithmetic	fadd, fsub, fmul
Fused	fmadd, fmsub, fnmsub, fnmadd
Divide / root	fdiv, fsqrt (iterative, multi-cycle)
Sign / select	fsgnj{,n,x}, fmin, fmax
Compare	feq, flt, fle
Convert	float $\leftrightarrow$ int (signed/unsigned), FP32 $\leftrightarrow$ FP16
Move / classify	fmv.x.~, fmv.~.x, fclass
Bit-exact checks vs IEEE reference: 116,089 (0 mismatches)	

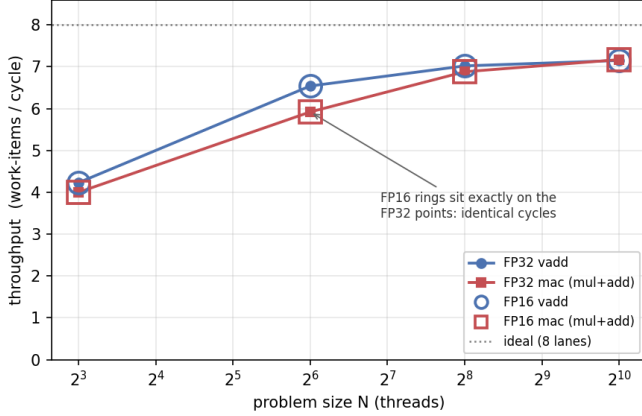


Fig. 2. Floating-point throughput versus problem size for streaming add and multiply-add kernels. The ramp is the same latency-hiding behavior as the integer suite; the hollow FP16 markers sit exactly on the FP32 curves because both use the same pipelined datapath.

fused $C_i = A_i B_i + C_i$, and $C_i = \sqrt{A_i/B_i}$ (two dependent multi-cycle ops per thread), all bit-exact to 1024 threads.

FP throughput and application kernels. The FP unit is pipelined (Section VI); an FP instruction commits a fixed number of cycles after issue and the issuing warp parks a single slot until it does, latency that SIMT interleaving hides on streaming work. Figure 2 plots throughput versus problem size for streaming add and multiply-add in both precisions: the same latency-hiding ramp as the integer suite, saturating near 7 work-items/cycle at 100% lane utilization, with the FP16 points lying exactly on the FP32 curves—half precision shares the pipelined datapath and costs no extra time. Table III runs four toolchain-assembled FP kernels through the engine. Two effects stand out: the iterative divide/square-root unit is visible ($f_{\text{norm}} = A/\sqrt{B}$ takes $\sim 6\times$ the cycles of streaming f_{saxpy} , the honest price of correctly-rounded SFU operations), and divergence interacts with the now-larger lane datapath (f_{dot} 's reduction leaves one of eight lanes active during the collapse, 45% utilization, so the absolute energy a mask-gated design saves grows with the FP-enlarged lane).

TABLE III

FP APPLICATION KERNELS (FP32 UNLESS NOTED), VERIFIED BIT-EXACT. CYCLES, GLOBAL LINE TRANSACTIONS, AND LANE UTILIZATION AT $N=256$.

Kernel	Computation	Cycles	Gmem	Lane-util
f_{saxpy}	$\alpha A_i + B_i$ (fmadd)	419	96	100%
f_{saxpy16}	$\alpha A_i + B_i$ (FP16)	451	96	100%
f_{norm}	$A_i/\sqrt{B_i}$ (div+sqrt)	2,396	96	100%
f_{dot}	$\sum_l A_{8w+l} B_{8w+l}$	1,991	96	45%

VI. PLACED TIMING CLOSURE OF THE FP DATAPATH

Floating point raises the accelerator from 30,088 to 74,342 LUTs ($2.5\times$), flip-flops from 4,590 to 7,894, and DSP blocks from 24 to 40 (the FP multipliers), at 1.34 W. The increase is dominated by the common-case arithmetic unit—eight per-lane copies of the add/multiply/FMA/convert datapath, the fused multiply-add the largest single block—not the iterative divide/square-root cores, which synthesize to only 689 LUTs each ($\sim 7\%$ of the design). The FMA is also the longest combinational chain: as a single-cycle block its raw path is 16.2 ns (61 MHz), making it the frequency bottleneck. Table IV summarizes the placed utilization and frequency of each step in the closure; we walk through them. In the table, steps 1–3 are out-of-context runs holding the lane count at eight, the DSE row trims to four lanes, and the final row is the complete SoC implemented in-context with full pin constraints, after the fetch/execute split described at the end of this section. LUTRAM is the distributed-RAM subset of the LUT count (in the SoC row it includes the eight-bank shared memory), the design uses zero block RAM throughout, and utilization percentages are of the `xczu7ev` (230,400 LUTs, 460,800 FFs, 1,728 DSPs).

Two-stage FMA (78 MHz, interconnect-bound). We first pipeline the FP unit into two stages, split at the FMA accumulator (mag) boundary; the warp pool absorbs the extra cycle with a single-slot scoreboard (`W_FPC`), the same park-and-resume mechanism used for memory and the SFU, so no new structural hazard arises and SIMT interleaving hides the latency. Timing-driven synthesis with retiming cuts the FP logic depth to 5.3 ns, yet place-and-route closes at 12.8 ns (**78 MHz**). The placed critical path is still the first FMA stage but now routing-dominated—7.4 of 12.8 ns (58%) is wire delay—because in the FP-enlarged fabric ($2.5\times$ the integer LUTs across eight wide FP lanes) interconnect, not logic depth, has become the binding constraint.

Confirming the diagnosis (84 MHz). If congestion is the limit, de-congesting the fabric should raise frequency with the logic unchanged. We fold the eight per-lane divide/square-root cores into a single shared serial unit, fed lane-by-lane. This removes $\sim 3.7\text{k}$ LUTs and $\sim 1.2\text{k}$ flip-flops and drops power from 1.53 to 1.22 W; crucially it leaves the FMA critical path structurally unchanged yet cuts that path's routing delay from 7.4 to 6.5 ns, raising the placed maximum to **84 MHz** and reducing near-critical endpoints from 8.7k to 3.2k. Same logic,

TABLE IV
PLACED POST-ROUTE PPA OF THE FP-ENABLED ACCELERATOR ACROSS THE TIMING-CLOSURE STEPS (#1–3), THE FOUR-LANE DSE POINT, AND THE FINAL DEPLOYABLE SoC (XCZU7EV, 100 MHz CONSTRAINT); THE ACCOMPANYING TEXT EXPLAINS HOW TO READ THE ROWS.

#	FP datapath	Lanes	LUT (31%)	LUTRAM	FF (3%)	DSP (2%)	BRAM	Power (W)	Fmax (MHz)	Crit. path
1	Two-stage FMA	8	72,573	3,348	12,347	40	0	1.53	78.0	FMA cone
2	+ shared serial SFU	8	68,914	3,348	11,153	40	0	1.22	84.1	FMA cone
3	Three-stage FMA	8	69,987	3,348	14,954	40	0	1.22	100.7	VRF LUTRAM
<i>DSE</i> : four lanes		4	36,785	1,748	6,953	20	0	0.98	79.7	FMA cone
<i>SoC</i> : deployable chip		8	71,567	7,244	17,834	40	0	1.20	103.7	FP ctl. broadcast

less congestion, higher frequency—a direct confirmation that interconnect, not logic depth, bounds this design.

Localizing the limit: frequency does not scale with fabric size. A natural question is whether *trimming* the FP fabric relieves congestion further. It does not. Placing the design at four lanes (Table IV, DSE row) halves its area—37k versus 69k LUTs, 20 versus 40 DSPs—but reaches only 79.7 MHz, marginally *below* the eight-lane 84.1 MHz, and the router still reports CLB congestion. The critical path is a *single* lane’s FMA cone, whose logic depth and locally-dense 128-bit datapath are independent of the lane count; halving the lanes replicates that cone fewer times but packs the smaller design into a denser region that gives the lone FMA no relief. The eight-lane configuration thus Pareto-dominates the four-lane one (equal frequency, twice the throughput) and identifies the true frequency lever as the FMA *microarchitecture*, not the fabric size.

Closing 100 MHz: a three-stage FMA. Acting on that finding, we split the FMA across *three* stages—product, 128-bit align-and-add, and normalize-and-round—so no single stage carries the whole cone. The warp pool absorbs the extra cycle with the same single-slot scoreboard, now a two-cycle writeback latency that SIMT interleaving continues to hide on streaming work; the added registers raise flip-flops to 15.0k while leaving LUTs (70.0k), DSPs, power (1.22 W), and the bit-exact results unchanged. Place-and-route now *meets* timing at 100 MHz (worst setup slack +0.071 ns; Table IV traces 78 → 84 → 100.7 MHz across the three steps), and decisively the critical path moves *off* the FMA onto the vector-register-file LUTRAM read mux—the same structure that bounds the *integer* design (Section III). The floating-point datapath is no longer the frequency limiter, confirming that FMA logic depth—the lever the design-space exploration identified—was the true constraint all along.

From out-of-context closure to a deployable SoC. Closing the accelerator out-of-context is not yet a chip: integrated into `chip_top` with the host core and the eight-bank shared memory, the FP-enlarged SoC first places at only 82.5 MHz. The worst path is again congestion-bound, but now through the warp scheduler’s *fetch–execute* cloud: instruction fetch from the asynchronous LUTRAM memory, decode, register-file read, execute, and writeback all completed in one combinational cycle, and 82% of the placed path was routing on its

two highest-fanout broadcast nets. Registering FP operands at issue, replicating the broadcast nets, and fully pipelining the DSP multiplies recover part of the gap (96.4 MHz) but leave the single-cycle cloud intact. Splitting that cloud with a fetch/execute pipeline register—the fetched instruction and its warp context are registered at a new stage boundary, and a per-warp in-flight bit preserves the scheduler’s re-issue semantics exactly—applies the SIMT latency-hiding argument at instruction grain: with two or more runnable warps the pipeline stays full, and the measured cycle cost is $\leq 0.3\%$ on 1024-element kernels. The full chip then places at 107.3 MHz unconstrained and, with every port pin-constrained for the ZCU104 (clock on a clock-capable pin) and a warning-free design-rule check—no waivers or severity downgrades—signs off at +0.360 ns worst setup slack (**103.7 MHz**), 71,567 LUTs (31%), 40 DSPs, zero BRAM, and 1.20 W (Table IV, final row): a deployable bitstream, not only a timing report.

VII. RELATED WORK

Open RISC-V GPGPU cores include Vortex [1], a full OpenCL-capable multi-core design, and Ventus [2], a PTX/Vortex-style GPGPU; both target performance and scale and are correspondingly large. Simty [3] demonstrates generalized SIMT execution of the RISC-V ISA, and Nyuzi [4] is a synthesizable vector GPGPU for graphics research. Open floating-point units such as FPnew [11] offer multifunction arithmetic as standalone IP; SIMTiX instead integrates a per-lane FP datapath into the SIMT pipeline and studies its *placed* timing. Relative to these, SIMTiX is deliberately small and readable, and its contribution is not raw performance but a complete, reproducible RTL-to-bitstream flow paired with a placed timing-closure study of a SIMT floating-point datapath—a result, the interconnect-bound nature of an FP-enlarged SIMT fabric and the FMA-microarchitecture lever that closes it, that the larger cores do not report. Control-divergence handling via reconvergence stacks is long studied [5], [6]; we use the standard stack mechanism and reuse its mask for lane clock-gating.

VIII. CONCLUSION

SIMTiX shows that the full SIMT mechanism set fits in a compact, fully-open RISC-V accelerator that runs end-to-end through a real implementation flow, with a distributed-RAM register file the key to making it small and fast on

FPGA. A per-lane floating-point unit adds FP32 and FP16 arithmetic—including the fused multiply-add and iterative divide/square root—with half precision obtained almost for free from the single-precision path and verified bit-exact over 116,089 vectors. The central result is a placed timing-closure study: the FP-enlarged fabric is interconnect-bound, a fact we diagnose, confirm by de-congestion, and localize to a single lane’s FMA cone, then close 100 MHz with a three-stage FMA that moves the critical path off the floating-point datapath onto the same register-file read mux that bounds the integer design. A final fetch/execute pipeline in the warp scheduler carries that closure from the out-of-context accelerator to the complete SoC, which signs off at 103.7 MHz with full ZCU104 pin constraints and a warning-free DRC—a deployable bitstream, at a measured cycle cost under 0.3%. An open-source ASIC path through the SkyWater 130 nm PDK [19] with OpenLane [20] would turn these FPGA numbers into silicon-relevant area, frequency, and power. The design and its evaluation infrastructure are public, so the results are reproducible and the platform is reusable for teaching and research.

REFERENCES

- [1] B. Tine, K. P. Yalamarthy, F. Elsabbagh, and H. Kim, “Vortex: Extending the RISC-V ISA for GPGPU and 3D-Graphics,” in *Proc. IEEE/ACM MICRO*, 2021.
- [2] F. Zhang *et al.*, “Ventus: a RISC-V based GPGPU,” open-source project, 2022.
- [3] S. Collange, “Simty: Generalized SIMT execution on RISC-V,” in *Workshop on Computer Architecture Research with RISC-V (CARRV)*, 2017.
- [4] J. Bush, P. Dexter, T. N. Miller, and A. Carpenter, “Nyami: a synthesizable GPU architectural model for general-purpose and graphics-specific workloads,” in *Proc. IEEE ISPASS*, 2015.
- [5] W. W. L. Fung, I. Sham, G. Yuan, and T. M. Aamodt, “Dynamic Warp Formation and Scheduling for Efficient GPU Control Flow,” in *Proc. IEEE/ACM MICRO*, 2007.
- [6] E. Lindholm, J. Nickolls, S. Oberman, and J. Montrym, “NVIDIA Tesla: A Unified Graphics and Computing Architecture,” *IEEE Micro*, vol. 28, no. 2, 2008.
- [7] T. M. Aamodt, W. W. L. Fung, and T. G. Rogers, *General-Purpose Graphics Processor Architectures*. Morgan & Claypool, 2018.
- [8] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.
- [9] A. Bakhoda, G. L. Yuan, W. W. L. Fung, H. Wong, and T. M. Aamodt, “Analyzing CUDA Workloads Using a Detailed GPU Simulator,” in *Proc. IEEE ISPASS*, 2009.
- [10] J. Leng *et al.*, “GPUWattch: Enabling Energy Optimizations in GPGPUs,” in *Proc. ACM/IEEE ISCA*, 2013.
- [11] S. Mach, F. Schuiki, F. Zaruba, and L. Benini, “FPnew: An Open-Source Multiformat Floating-Point Unit Architecture for Energy-Proportional Transprecision Computing,” *IEEE Trans. VLSI Syst.*, vol. 29, no. 4, pp. 774–787, 2021.
- [12] A. Waterman and K. Asanović, “The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA,” RISC-V International, 2019.
- [13] RISC-V International, “RISC-V “V” Vector Extension, Version 1.0,” specification, 2021.
- [14] IEEE, “IEEE Standard for Floating-Point Arithmetic,” *IEEE Std 754-2019*, 2019.
- [15] J.-M. Muller *et al.*, *Handbook of Floating-Point Arithmetic*, 2nd ed. Birkhäuser, 2018.
- [16] M. D. Ercegovac and T. Lang, *Digital Arithmetic*. Morgan Kaufmann, 2004.
- [17] S. A. Figueroa, “When is double rounding innocuous?,” *ACM SIGNUM Newsletter*, vol. 30, no. 3, pp. 21–26, 1995.
- [18] W. Snyder, “Verilator: Open-Source SystemVerilog Simulation and Verification,” Veripool, <https://www.veripool.org/verilator>.
- [19] SkyWater Technology and Google, “SkyWater Open Source PDK (sky130),” <https://github.com/google/skywater-pdk>.
- [20] A. Ghazy and M. Shalan, “OpenLane: The Open-Source Digital ASIC Implementation Flow,” in *Workshop on Open-Source EDA Technology (WOSET)*, 2020.